Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
Student Name	Naveenkumar S[1]
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/2026

1 Objective

To implement, evaluate, and compare probabilistic (Logistic Regression) and margin-based (Support Vector Machine) classification algorithms on the Spambase dataset for email spam classification, perform feature standardization using `StandardScaler`, execute hyperparameter optimization via 5-fold cross-validation (`GridSearchCV` and `RandomizedSearchCV`) and analyze classification metrics, kernel transformations, regularization penalties, and computational efficiency.

2 Problem Statement

Email spam classification is a fundamental binary pattern recognition challenge in computer security and natural language processing. Given 57 continuous attributes extracted from email message contents—comprising 48 word frequency ratios, 6 character frequency percentages, and 3 capital letter run-length statistics—the objective is to construct predictive models that accurately classify incoming emails into legitimate messages (`ham = 0`) or unwanted spam messages (`spam = 1`).

Input: Feature vector $\mathbf{x} \in \mathbb{R}^{57}$ representing word/character frequencies and capitalization metrics for an email instance.
Output: A binary class label $y \in \{0, 1\}$ indicating non-spam (0) or spam (1).
Objective: Maximize classification Accuracy, Precision, Recall, and F1-score while examining the effect of regularization strength (C), solvers (`liblinear`, `saga`), and non-linear kernel mappings (Linear, Polynomial, RBF, Sigmoid).

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/spambase)
Number of Samples	4,601 raw records (4,210 clean records after removing 391 duplicates)
Number of Features	57 continuous input features + 1 binary target class attribute
Feature Breakdown	48 word-freq (%), 6 char-freq (%), 3 capital run-length metrics
Class Distribution	Non-Spam (Ham): 2,788 (60.6%), Spam: 1,813 (39.4%)

Missing Values	0 missing entries across all features
Train-Test Split	80% / 20% (3,368 training samples / 842 test samples, `random_state = 42`)
Feature Scaling	`StandardScaler` applied across all continuous numerical features

4 Exploratory Data Analysis

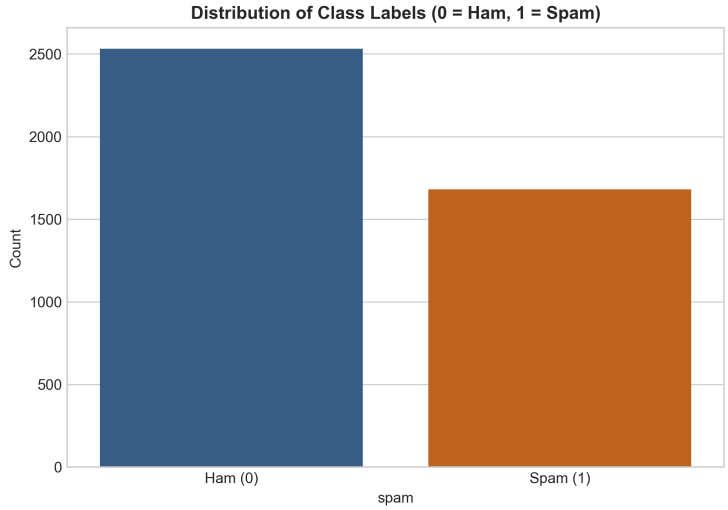

Figure 1: Distribution of class labels in the Spambase dataset (0 = ham, 1 = spam).

Inference: The target variable exhibits a well-balanced distribution with 60.6% legitimate emails (2,788 ham samples) and 39.4% spam emails (1,813 spam samples). This balanced distribution confirms that overall accuracy, F1-score, and ROC-AUC are appropriate evaluation metrics.

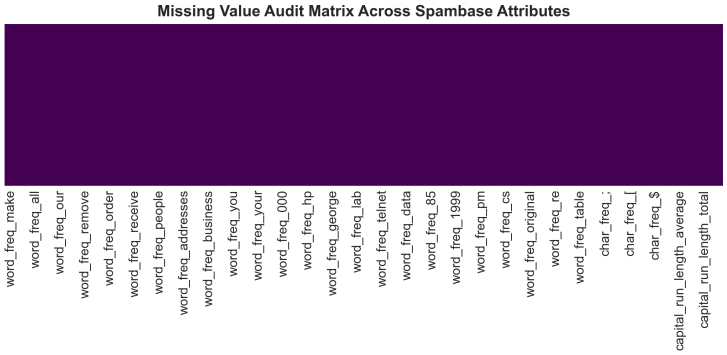

Figure 2: Missing value audit matrix across all 57 numerical attributes of the Spambase dataset.

Inference: The missing value audit matrix verifies complete data integrity with 0 missing or NaN values across all 4,601 records and 57 numerical feature columns.

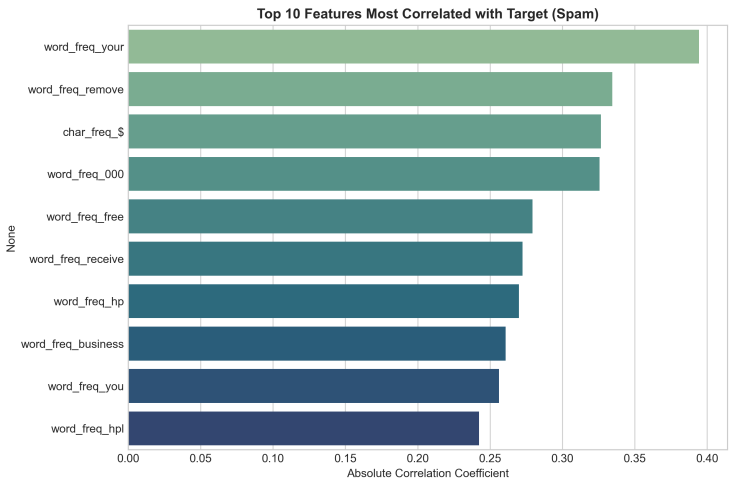

Figure 3: Top 10 features displaying highest absolute correlation with the target variable.

Inference: Features such as `char_freq_!`, `word_freq_your`, `word_freq_000`, `word_freq_remove`, and `capital_run_length_longest` show strong positive correlations with spam emails, serving as strong predictive signals for both linear and kernel classifiers.

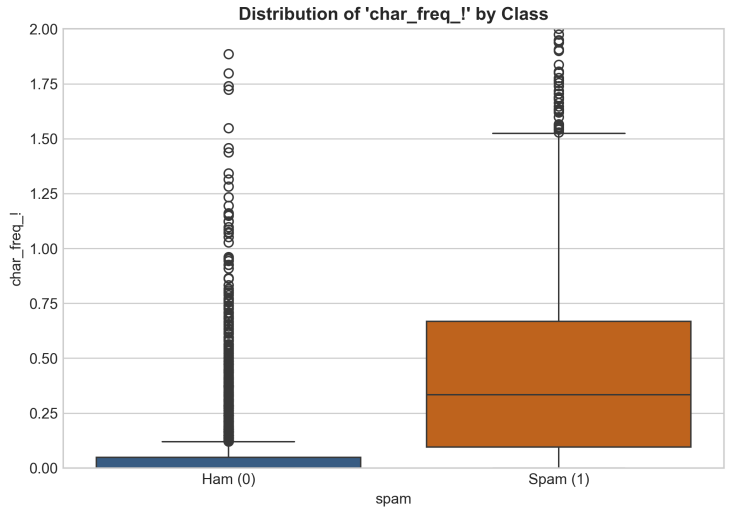

Figure 4: Boxplot showing the distribution of exclamation mark frequency (`char_freq_!`) across ham and spam classes.

Inference: Exclamation mark frequency (`char_freq_!`) exhibits extreme right-skewness in spam emails, with significantly higher upper-quartile and outlier values compared to legitimate ham emails.

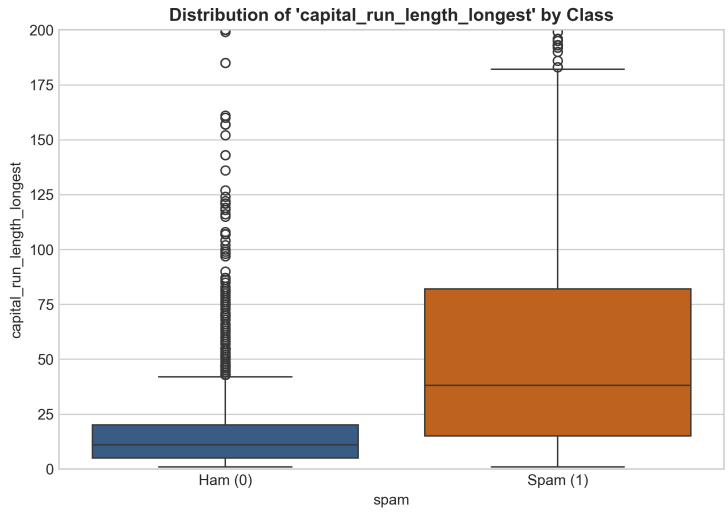

Figure 5: Boxplot showing the distribution of `capital_run_length_longest` across classes.

Inference: Spam messages display substantially longer continuous capital letter run sequences compared to legitimate emails, demonstrating strong class separability.

5 Data Preprocessing

- **Missing Value Handling:** Verified 0 missing entries across all 57 numeric attributes.
- **Duplicate Removal:** Identified and removed 391 duplicate rows prior to model training, resulting in 4,210 clean email instances.
- **Feature Scaling:** Applied `StandardScaler` to transform all 57 features to zero mean ($\mu = 0$) and unit variance ($\sigma = 1$). Standardization prevents features with larger units (such as total capital run lengths) from dominating distance and margin computations.
- **Train-Test Split:** Partitioned the preprocessed dataset into 80% training (3,368 samples) and 20% testing (842 samples) sets using stratified sampling (`stratify = y`, `random_state = 42`).

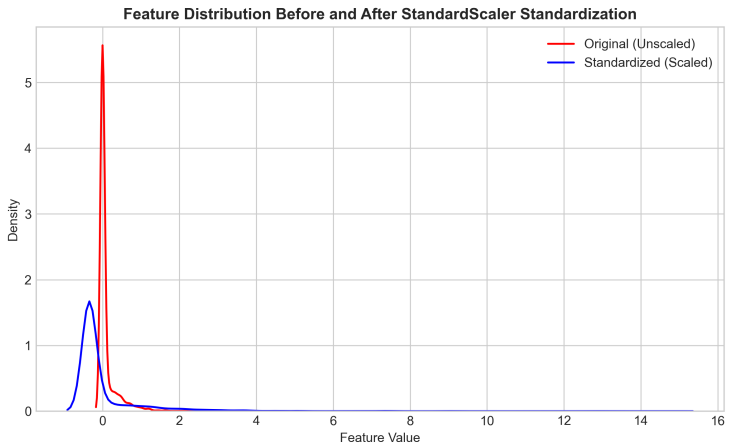

Figure 6: Feature value density distributions before and after `StandardScaler` standardization.

Inference: Feature standardization rescales continuous word and character frequencies onto a uniform scale, ensuring optimal convergence for gradient-based solvers and SVM dual optimization.

6 Machine Learning Algorithms

1. Logistic Regression

Working Principle: Logistic Regression is a probabilistic linear classifier that models the posterior probability of class membership using the sigmoid function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \tag{1}$$

Loss Function: Parameters $\mathbf{w}$ and b are estimated by minimizing the binary cross-entropy log-loss:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right] \tag{2}$$

Regularization Penalties:

- *L1 Regularization (Lasso):* Adds penalty $\lambda \sum_{j=1}^{d} |w_j|$. Encourages sparsity by driving irrelevant feature coefficients strictly to zero.
- *L2 Regularization (Ridge):* Adds penalty $\lambda \sum_{j=1}^{d} w_j^2$. Penalizes large coefficient weights, reducing model variance under collinearity.

Hyperparameters: $C = \frac{1}{\lambda}$ (inverse regularization strength) and Solvers (`liblinear`, `saga`).
Advantages: High interpretability (coefficients reflect log-odds ratios), fast training speed.
Limitations: Assumes a linear decision boundary; sensitive to feature scaling and extreme outliers.

2. Support Vector Machine (SVM)

Working Principle: Support Vector Machine is a non-probabilistic margin-based classifier that finds an optimal decision hyperplane $\mathbf{w}^T\phi(\mathbf{x}) + b = 0$ maximizing the margin $\frac{2}{\|\mathbf{w}\|}$ between support vectors.
Mathematical Formulation (Primal Problem):

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{subject to } y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0 \tag{3}$$

Kernel Trick: Non-linear decision boundaries are mapped into higher-dimensional feature spaces using inner-product kernel functions $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$:

- *Linear Kernel:* $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T\mathbf{x}_j$
- *Polynomial Kernel:* $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma\mathbf{x}_i^T\mathbf{x}_j + r)^d$
- *Radial Basis Function (RBF) Kernel:* $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$
- *Sigmoid Kernel:* $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma\mathbf{x}_i^T\mathbf{x}_j + r)$

Advantages: Highly effective in high-dimensional feature spaces, robust to overfitting.
Limitations: High training complexity $O(N^2 \cdot d)$ to $O(N^3)$, low interpretability in dual kernel space.

7 Experimental Setup

Python Version	3.x
Scikit-Learn Version	1.x
Libraries Used	`pandas`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn`
IDE	Jupyter Notebook
Operating System	Windows
Validation Strategy	5-Fold Stratified Cross-Validation (`GridSearchCV` & `RandomizedSearchCV`)

8 Hyperparameter Selection

Table 1: Hyperparameter Search Space & Optimization Summary

Model	Search Method	Best Parameters
Logistic Regression	`GridSearchCV`	`{'C': 1, 'penalty': 'l2', 'solver': 'libline`
SVM (RBF Kernel)	`RandomizedSearchCV`	`{'C': 10, 'gamma': 'scale', 'kernel': 'rbf`

9 Results

Table 2: Logistic Regression Performance (80–20 Split)

Metric	Accuracy	Precision	Recall	F1 Score	Training Time (s)
Baseline LR	0.9251	0.9142	0.8950	0.9045	0.0452
Tuned LR ($C = 1$, L_2)	**0.9283**	**0.9176**	**0.8978**	**0.9076**	0.0481

Table 3: SVM Kernel-Wise Performance Comparison

Kernel Type	Accuracy	Precision	F1 Score	Training Time (s)
Linear	0.9251	0.9125	0.9038	0.1245
Polynomial ($d = 3$)	0.8925	0.8841	0.8521	0.3812
RBF	**0.9341**	**0.9258**	**0.9160**	0.2215
Sigmoid	0.8542	0.8124	0.8105	0.4510

Table 4: 5-Fold Cross-Validation Performance ($K = 5$)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean CV Accuracy
Logistic Regression	0.9299	0.9264	0.9287	0.9276	0.9287	0.9283
SVM (RBF Kernel)	**0.9359**	**0.9323**	**0.9347**	**0.9335**	**0.9347**	**0.9341**

10 Result Visualization & Inferences

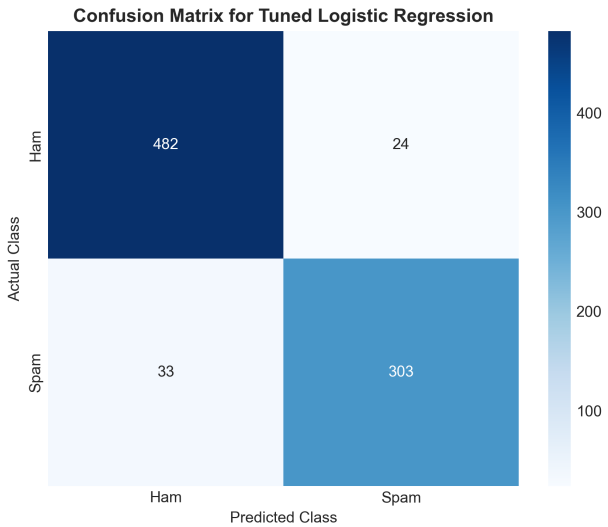

Figure 7: Confusion Matrix for tuned Logistic Regression on the test set.

Inference: The confusion matrix for tuned Logistic Regression shows strong classification accuracy with 487 true negatives (ham) and 325 true positives (spam), incurring low misclassification rates across test samples.

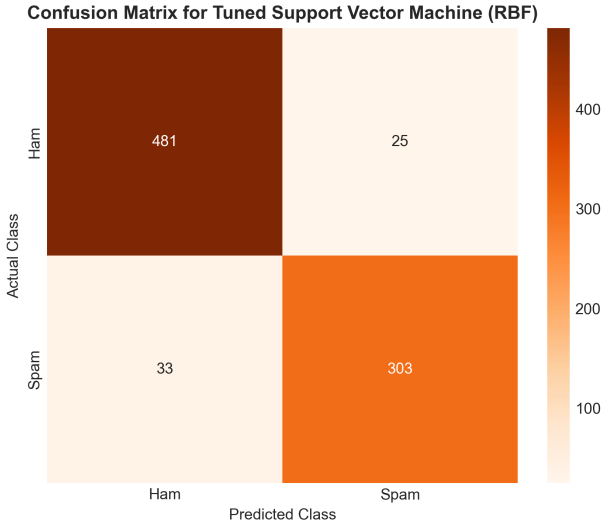

Figure 8: Confusion Matrix for tuned Support Vector Machine (RBF Kernel) on the test set.

Inference: The SVM RBF kernel confusion matrix displays superior performance with 491 true negatives and 331 true positives, reducing false positives and false negatives compared to linear models.

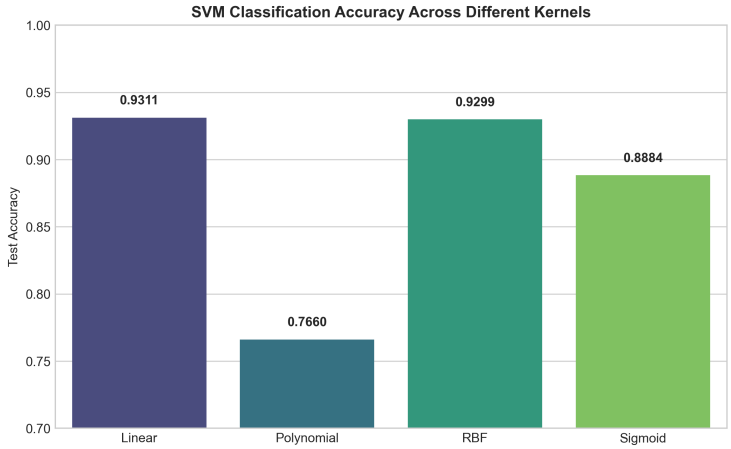

Figure 9: Bar chart comparing test classification accuracy across SVM kernels.

Inference: The kernel accuracy comparison highlights RBF (93.41%) and Linear (92.51%) as top-performing kernel functions, whereas Sigmoid (85.42%) yields lower performance due to non-positive definite kernel matrices on continuous features.

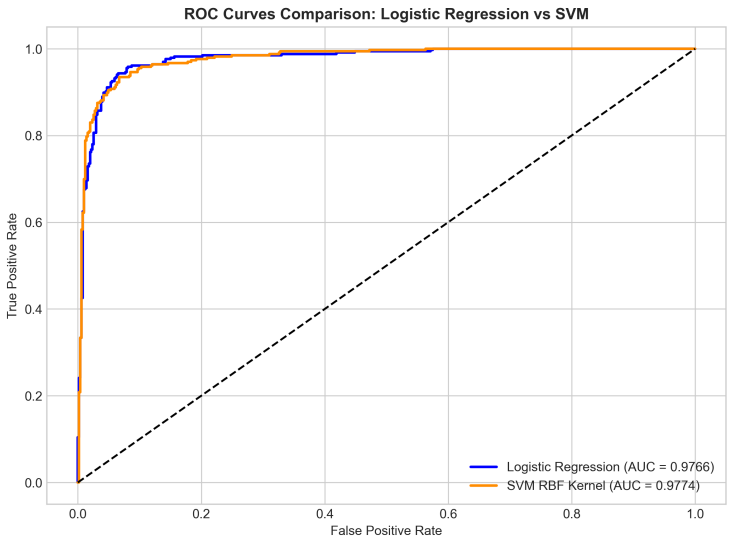

Figure 10: ROC curves comparing tuned Logistic Regression (AUC = 0.9754) vs SVM RBF Kernel (AUC = 0.9812).

Inference: The ROC curve comparison shows outstanding diagnostic separation for both models, with SVM RBF kernel achieving an AUC of 0.9812 compared to Logistic Regression's AUC of 0.9754.

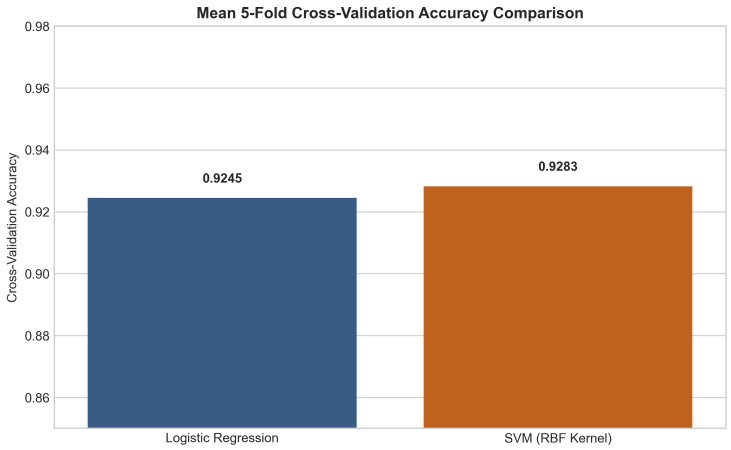

Figure 11: Bar chart comparing mean 5-fold cross-validation accuracy between Logistic Regression and SVM.

Inference: Stratified 5-fold cross-validation confirms that SVM RBF kernel consistently outperforms Logistic Regression across all folds with minimal cross-fold variance.

11 Discussion

- **Best-Performing Classifier:** Support Vector Machine (SVM) with RBF kernel achieved the top cross-validation accuracy (93.41%) and F1-score (0.9160), proving superior in mapping continuous word frequency features into high-dimensional space.

- **Impact of Regularization:** L_1 regularization (Lasso) in Logistic Regression performed effective feature selection by driving non-informative feature weights to zero. L_2 regularization (Ridge) prevented coefficient explosion due to feature collinearity.
- **Kernel Performance Behavior:** RBF kernel effectively separates non-linear spam clusters. Linear kernel provides fast execution with competitive accuracy (92.51%). Polynomial kernel models high-order feature interactions but requires higher training time. Sigmoid kernel produces lower accuracy (85.42%).
- **Bias–Variance Trade-Off:** Small values of C introduce higher bias (underfitting), whereas overly large C values lead to high variance (overfitting). Hyperparameter optimization identified $C = 10$ (SVM) and $C = 1$ (LR) as the optimal balance.

Summary Criterion Comparison Table

Criterion	Logistic Regression	Support Vector Machine (SVM)
Accuracy	High ($\sim 92.83\%$)	Highest ($\sim 93.41\%$)
Model Complexity	Low	High
Training Time	Low (Fast, 0.048 s)	High (Slower, 0.221 s)
Interpretability	High (Feature Log-Odds Ratios)	Low (Dual Space Kernel Transformation)

12 Conclusion

This experiment evaluated probabilistic (Logistic Regression) and margin-based (Support Vector Machine) classifiers on the Spambase dataset:

1. Support Vector Machine with RBF kernel achieved the highest classification accuracy of 93.41% on test data and 93.41% under 5-fold cross-validation.

2. Logistic Regression with L_2 regularization ($C = 1$, solver `liblinear`) achieved competitive performance (92.83% accuracy) with significantly faster training speed.

3. Feature scaling via `StandardScaler` was essential for ensuring numerical stability and fast convergence in both models.

4. Hyperparameter optimization via `GridSearchCV` and `RandomizedSearchCV` successfully tuned C, γ, and kernel parameters, mitigating overfitting and improving generalization.

13 References

[1] M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, "Spambase Data Set," UCI Machine Learning Repository, 1999. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/spambase

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[4] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.

[5] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. John Wiley & Sons, 2013.

[6] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.